Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к лабораторной работе №4
на тему:
«УПРАВЛЕНИЕ ПРОЦЕССАМИ И ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Написать программу в соответствии с вариантом задания, обеспечить её демонстрацию и подготовить сценарий сборки проекта с помощью make.

Самовосстанавливающийся процесс: при получении сигнала, стандартно вызывающего завершение, процесс должен создать копию, которая продолжит выполнение с прерванного места, после чего исходный процесс завершает работу.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В рамках лабораторной работы разработана программа self_healing.c, моделирующая самовосстанавливающийся процесс. В качестве наглядной полезной нагрузки используется счётчик, значение которого через равные интервалы времени записывается в журнал self_healing.log. Такая форма демонстрации позволяет легко наблюдать работу процесса и проверять его устойчивость к сигналам завершения.

Для обработки сигналов SIGTERM и SIGINT используется механизм sigaction() [1]. Обработчик сигнала не завершает процесс немедленно, а только устанавливает глобальный флаг restart_requested. Это решение делает обработку асинхронного события безопасной и переносит основную логику восстановления в обычный поток выполнения программы.

В основном цикле программа увеличивает счётчик, записывает в журнал идентификатор процесса и текущее значение counter, а затем вызывает sleep(). Если после очередной итерации обнаруживается установленный флаг restart_requested, процесс порождает дочерний процесс с помощью fork() [2]. Родительский процесс завершается, а дочерний наследует состояние памяти и продолжает выполнение программы.

Для удобства работы создан Makefile с целями all, run и clean. Цель run запускает программу в фоновом режиме, а журнал self_healing.log используется как простой протокол, позволяющий визуально наблюдать непрерывность работы и смену идентификатора процесса после восстановления.

На рисунке 2.1 показаны первые записи журнала, соответствующие нормальной работе процесса до получения сигнала. Видно, что приложение периодически сохраняет свой PID и значение счётчика, что подтверждает его активность и корректную организацию цикла выполнения.

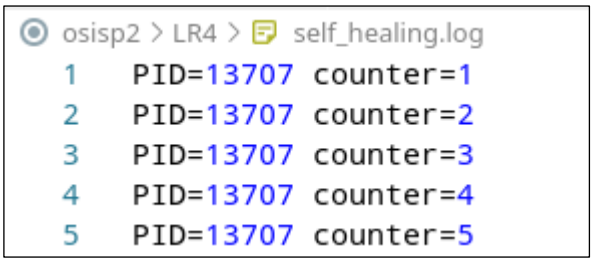


Рисунок 2.1 – Фрагмент журнала до отправки сигнала завершения

После отправки сигнала SIGTERM исходному процессу, как показано на рисунке 2.2, был создан дочерний процесс с новым PID 13776. В журнале видно, что после строк PID=13707 counter=1, PID=13707 counter=2 и т.д. появились записи с PID=13776. Это означает, что логическое состояние вычислений было сохранено, а выполнение продолжилось уже в новом процессе [3].

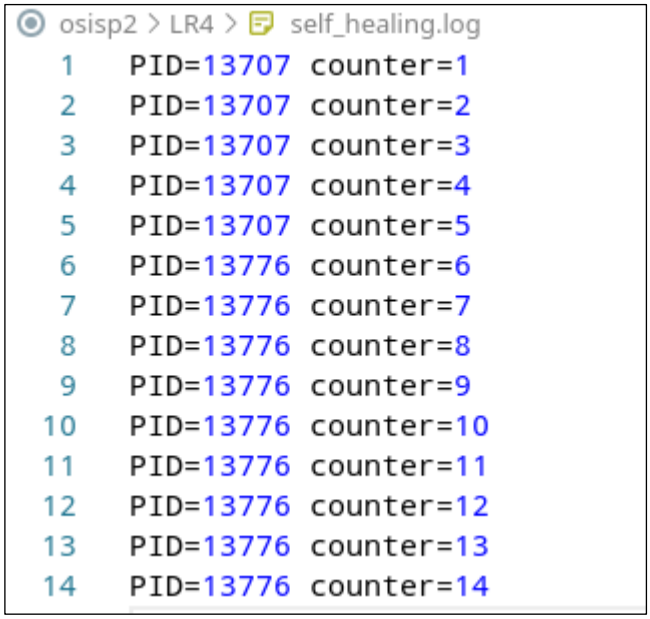


Рисунок 2.2 – Продолжение работы после восстановления процесса

Благодаря тому, что дочерний процесс наследует все данные родителя, включая текущее значение счётчика и открытые файловые дескрипторы, логическое состояние вычислений сохраняется полностью. Журнал self_healing.log служит простым и наглядным протоколом: в каждой записи указаны PID текущего процесса и значение счётчика, что позволяет визуально убедиться в смене PID без потери прогресса. Makefile, приведённый в листинге А.2, поддерживает цели all (сборка исполняемого файла self_healing), clean (удаление исполняемого файла и журнала) и run (запуск программы в фоновом режиме), что автоматизирует типовые действия разработчика и тестировщика.

ВЫВОД

В результате выполнения лабораторной работы были достигнуты цели лабораторной работы. Изучены основные особенности подсистемы управления процессами в Unix: порождение процессов с помощью fork(), обработка сигналов с использованием надёжного механизма sigaction(), завершение процессов и передача кода завершения. Реализован самовосстанавливающийся процесс, который при получении сигнала SIGTERM или SIGINT не завершается, а создаёт дочерний процесс, наследующий состояние родителя, и продолжает выполнение с прерванного места. В качестве демонстрации «живости» процесса использован счётчик, периодически записывающий своё значение вместе с PID в файл self_healing.log, что позволяет наблюдать непрерывность работы и факт восстановления. Программа корректно обрабатывает асинхронные сигналы с помощью флага volatile sig_atomic_t, избегая состояния гонки. Для автоматизации сборки и тестирования создан Makefile с целями all, clean и run. Таким образом, все задачи лабораторной работы выполнены в полном объёме, приобретены практические навыки управления процессами и взаимодействия процессов в среде Unix с использованием системных вызовов fork(), sigaction() и механизма сигналов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Linux manual page sigaction(2) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man2/sigaction.2.html. – Дата доступа: 11.02.2026.

[2] Linux manual page fork(2) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man2/fork.2.html. – Дата доступа: 11.02.2026.

[3] Linux manual page signal(7) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man7/signal.7.html. – Дата доступа: 11.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация самовосстанавливающегося процесса

```
#define _POSIX_C_SOURCE 200809L

#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>

#define LOGFILE "self_healing.log"

volatile sig_atomic_t restart_requested = 0;

void handle_signal(int sig)
{
    if (sig == SIGTERM || sig == SIGINT)
    {
        restart_requested = 1;
    }
}

void log_state(pid_t pid, int counter)
{
    int fd = open(LOGFILE, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd < 0)
        return;

    char buf[128];
    int len = snprintf(buf, sizeof(buf),
                       "PID=%d counter=%d\n", pid, counter);

    write(fd, buf, len);
    close(fd);
}

int main()
{
    struct sigaction sa;
    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;

    if (sigaction(SIGTERM, &sa, NULL) == -1)
    {
        perror("sigaction");
        exit(EXIT_FAILURE);
    }
    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction");
        exit(EXIT_FAILURE);
    }

    int counter = 0;
    pid_t pid = getpid();
```

```
    while (1)
    {
        counter++;
        log_state(pid, counter);
        sleep(3);

        if (restart_requested)
        {
            pid_t child = fork();

            if (child < 0)
            {
                _exit(EXIT_FAILURE);
            }

            if (child == 0)
            {
                pid = getpid();
                restart_requested = 0;
                continue;
            }
            else
            {
                _exit(EXIT_SUCCESS);
            }
        }
    }
}

/*
make run
ps -f | grep self_healing

kill -TERM <pid>
ps -f | grep self_healing

pkill -9 -f self_healing
make clean
*/
```

Листинг А.2 – Файл сборки проекта

```
CC=gcc
TARGET = self_healing
SRC = self_healing.c

all: $(TARGET)

$(TARGET): $(SRC)
    $(CC) $(SRC) -o $(TARGET)

clean:
    rm -f $(TARGET) self_healing.log

run: $(TARGET)
    ./$(TARGET) &
```